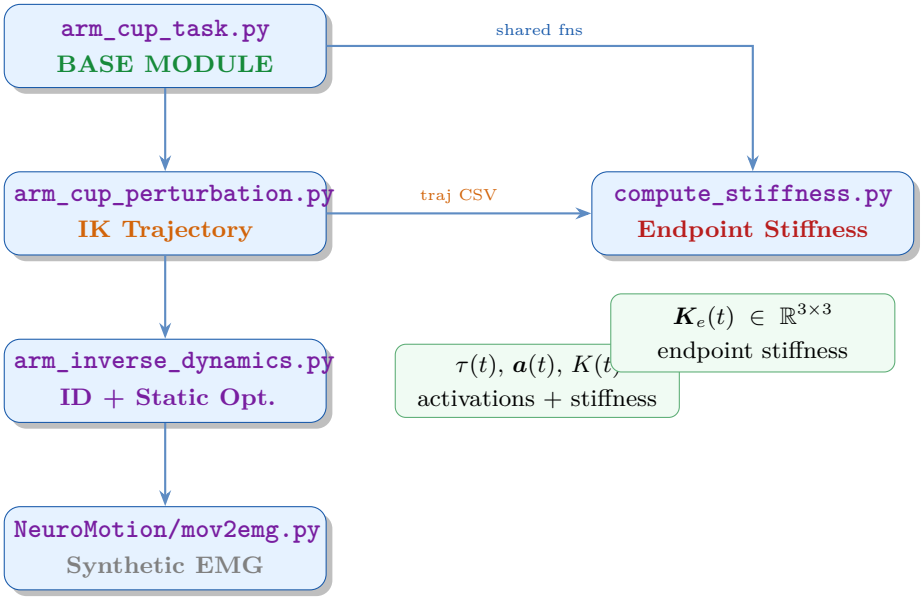
Ball-in-Cup EMG Synthetic Data Generator Pipeline

Comprehensive Script Documentation
with Equations, Examples and Output Plots



Ball-in-Cup Experimental Platform

Tele-Impedance / EMG Ground Truth Generation

May 30, 2026

Contents

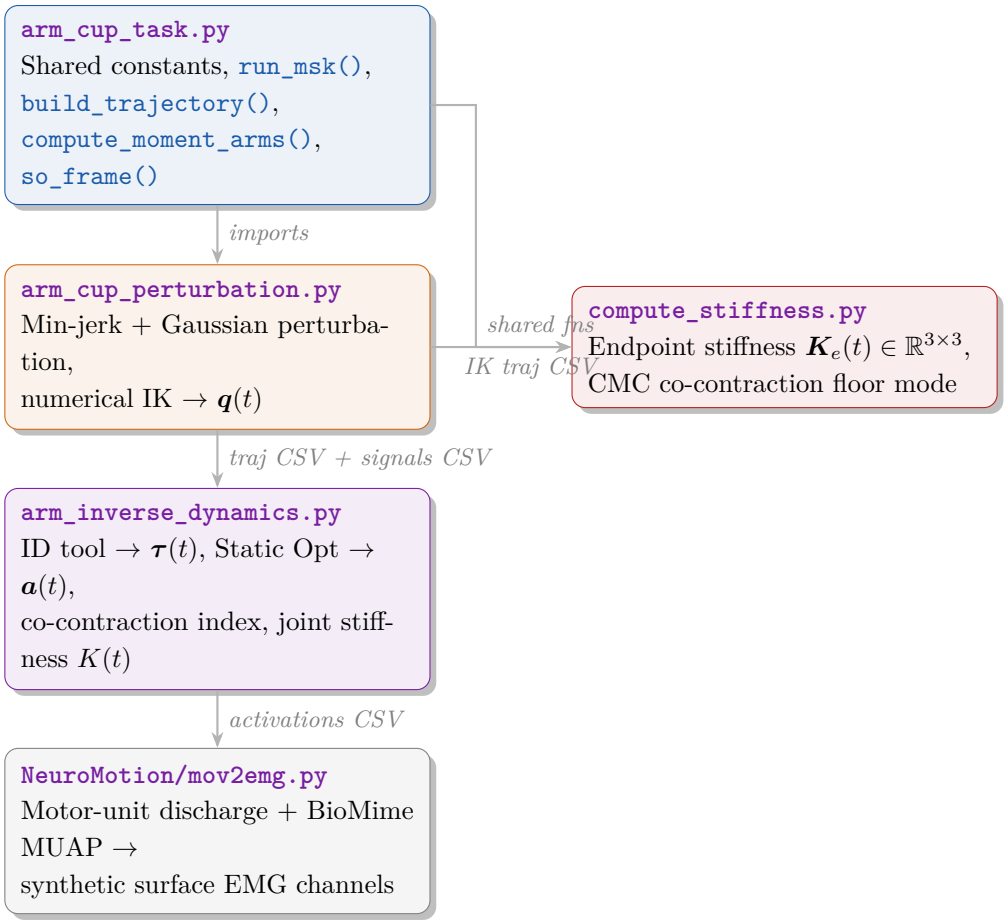
1 Project Overview and Pipeline

This document describes the full Python pipeline that generates musculoskeletal ground-truth data (joint torques, muscle activations, fiber lengths, joint and endpoint stiffness/damping) for the ball-in-cup reaching task. The data drives synthetic EMG generation via the NeuroMotion framework and serves as a reference for learning impedance behaviours in the SEA elbow exosuit controller.

Class-based architecture (current)

Every pipeline script now exposes a single top-level class wrapping its end-to-end behaviour, and all simulation-wide constants live in a centralised `config.py` (single source of truth). Each class can be driven via `ClassName().run()` (replicates the legacy CLI behaviour) or step-by-step. Backwards compatibility is preserved: every legacy module-level function (`build_trajectory`, `run_msk`, `compute_moment_arms`, `so_frame`, ...) remains importable. See Sec. 8 for the full class API and the centralised config map.

1.1 File Dependency Graph



1.2 Output Directory Layout

Script	Class	Output subdirectory
arm_cup_task.py	ArmCupTask	demo_output/arm_cup_task/
arm_cup_perturbation.py	ArmCupPerturbation	demo_output/arm_cup_perturbation/
arm_inverse_dynamics.py	ArmInverseDynamics	demo_output/arm_inverse_dynamics/
compute_stiffness.py	ComputeStiffness	demo_output/compute_stiffness/
compare_to_razavian2021.py	RazavianComparison	demo_output/compare_razavian/

All constants are imported from config.py; no script redeclares shared values locally.

1.3 Execution Sequence

Each script can be invoked from the command line (legacy entry point) or driven programmatically through its public class.

Run order - CLI

```
# Step 1 -- IK-solved trajectory with mechanical perturbation
python arm_cup_perturbation.py

# Step 2 -- Inverse dynamics + static optimisation + joint stiffness
python arm_inverse_dynamics.py

# Step 3 -- Endpoint stiffness K_e(t) and damping D_e(t)
python compute_stiffness.py --mode perturb --stiffness static_opt
# or with CMC co-contraction floor:
python compute_stiffness.py --mode perturb --stiffness cmc

# Step 4 -- Quantitative comparison vs. Razavian et al. ICRA 2021
python compare_to_razavian2021.py
```

Run order - Class API (equivalent)

```
from arm_cup_perturbation    import ArmCupPerturbation
from arm_inverse_dynamics    import ArmInverseDynamics
from compute_stiffness       import ComputeStiffness
from compare_to_razavian2021 import RazavianComparison

ArmCupPerturbation().run()                                  # Step 1
ArmInverseDynamics(tag="perturb").run()                     # Step 2
ComputeStiffness(mode="perturb", stiffness="cmc").run()     # Step 3
RazavianComparison().run()                                  # Step 4
```

1.4 Notation and Symbol Table

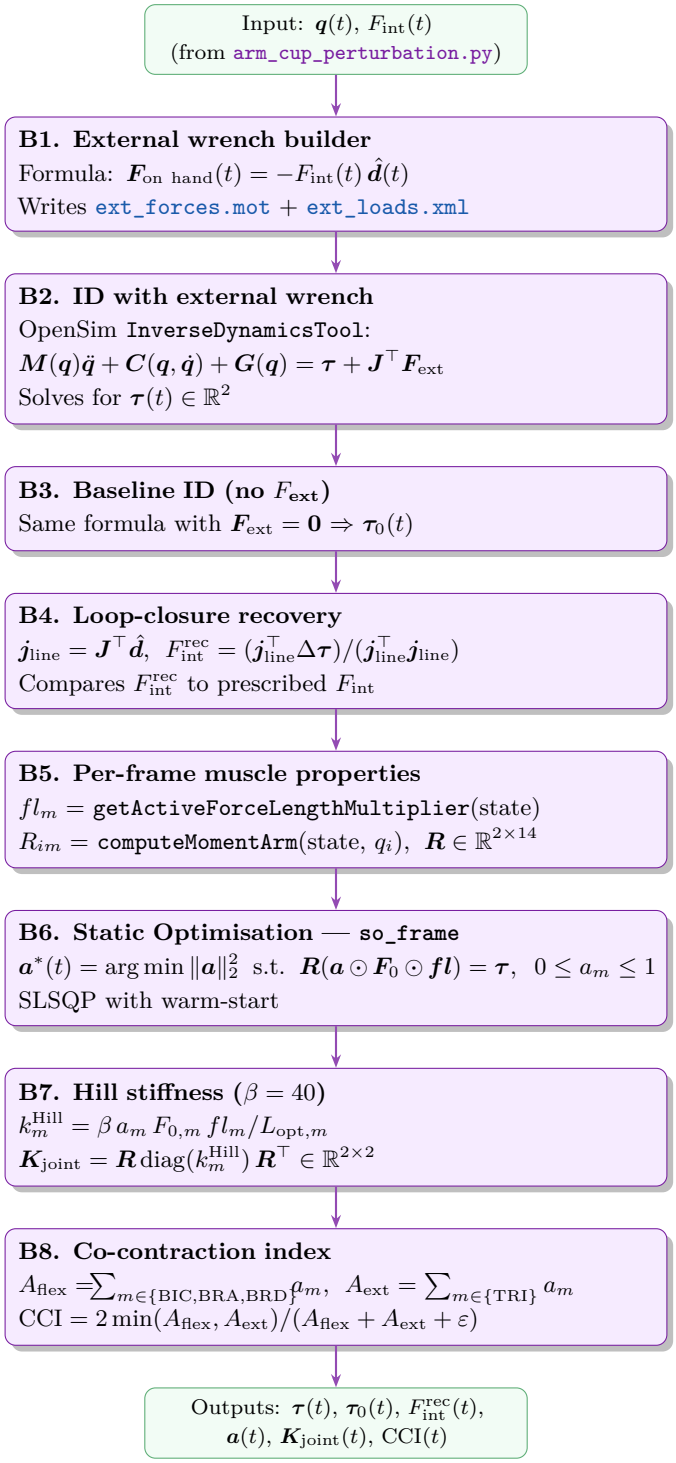
The following symbols are used consistently throughout all four scripts and this document. $m \in \{1 \dots 14\}$ indexes muscles; $i \in \{x, y, z\}$ indexes Cartesian components; j or k indexes generalised coordinates. $n_{\text{dof}} \in \{2, 7\}$ depending on context: arm_inverse_dynamics.py operates on the 2 IK-active DOFs; compute_stiffness.py uses all 7 active DOFs.

Symbol	Shape	Description	Un
$\boldsymbol{q}(t)$	$\mathbb{R}^7$	Generalised joint coordinate vector	rad
$\dot{\boldsymbol{q}}(t)$	$\mathbb{R}^7$	Joint angular velocities	rad/
$\ddot{\boldsymbol{q}}(t)$	$\mathbb{R}^7$	Joint angular accelerations	rad/
$\boldsymbol{\tau}(t)$	$\mathbb{R}^{n_{\text{dof}}}$	Generalised joint torques (output of ID)	N·m
$\boldsymbol{a}(t)$	$\mathbb{R}^{14}$	Muscle activation vector (output of SO), $a_m \in [0,1]$	–
$\boldsymbol{R}(t)$	$\mathbb{R}^{n_{\text{dof}}\times 14}$	Moment arm matrix: $R_{im} = \partial l_m / \partial q_i$	m/r
$\boldsymbol{F}_0$	$\mathbb{R}^{14}$	Maximum isometric muscle forces (model constants)	N
$\boldsymbol{fl}(t)$	$\mathbb{R}^{14}$	Active force–length multiplier per muscle	–
$l_m(t)$	scalar	Current fiber length of muscle m	m
$L_{\text{opt},m}$	scalar	Optimal fiber length of muscle m (model const.)	m
$\tilde{l}_m = l_m / L_{\text{opt}}$	scalar	Normalised fiber length (= 1 at peak force)	–
$k_m(t)$	scalar	Linearised muscle stiffness (Hill model)	N/r
$\boldsymbol{J}(t)$	$\mathbb{R}^{3\times n_{\text{dof}}}$	Hand endpoint Jacobian: $J_{ij} = \partial p_i / \partial q_j$	m/r
$\boldsymbol{J}^+$	$\mathbb{R}^{n_{\text{dof}}\times 3}$	Moore–Penrose pseudoinverse of $\boldsymbol{J}$	rad/
$\boldsymbol{K}_{\text{joint}}(t)$	$\mathbb{R}^{n_{\text{dof}}\times n_{\text{dof}}}$	Joint stiffness tensor	N·m
$\boldsymbol{K}_e(t)$	$\mathbb{R}^{3\times 3}$	Cartesian endpoint stiffness tensor	N/r
$\boldsymbol{D}_e(t)$	$\mathbb{R}^{3\times 3}$	Cartesian endpoint damping tensor	N·s/
$d_m(t)$	scalar	Hill linearised muscle damping (per muscle)	N·s/
β_D	0.3	Hill f-v slope coefficient near $v_m = 0$ (Zajac 1989)	–
$V_{\text{max,factor}}$	10 s^{-1}	$v_{\max} = V_{\text{max,factor}} \cdot l_{\text{opt}}$	1/s
$\lambda_{\min}$, $\lambda_{\max}$	scalar	Eigenvalues of $\boldsymbol{K}_e[0:2, 0:2]$ (XY sub-block)	N/r
$P_{\text{null}}(t)$	scalar	Null-space co-contraction proxy: $\min_m a_m$	–
$c_{\text{exo}}(t)$	scalar	SEA exo-cable pre-tension setpoint (driven by P_{null})	m
$F_{\text{int}}(t)$	scalar	Interaction force at hand–cup interface	N
$\hat{\boldsymbol{d}}$	$\mathbb{R}^3$	Unit direction of hand motion (ground frame)	–
β	40	Hill stiffness dimensionless scaling coefficient (`arm_inverse_dynamics.py` only)	–
γ	0.45	Gaussian force–length curve width parameter	–
α_{floor}	0.15 (CMC)	SLSQP lower bound on all activations in perturbation window	–
M_{eff}	2.0 kg	Effective inertial mass of cup + ball	kg
D	0.30 m	Reach distance along the rail	m
T	1.60 s	Reach duration	s
T_{pert}	0.90 s	Centre of Gaussian perturbation	s
σ	0.030 s	Width (std dev) of Gaussian perturbation	s
$\tau = t/T$	scalar	Normalised time within a movement segment, $\tau \in [0,1]$	–

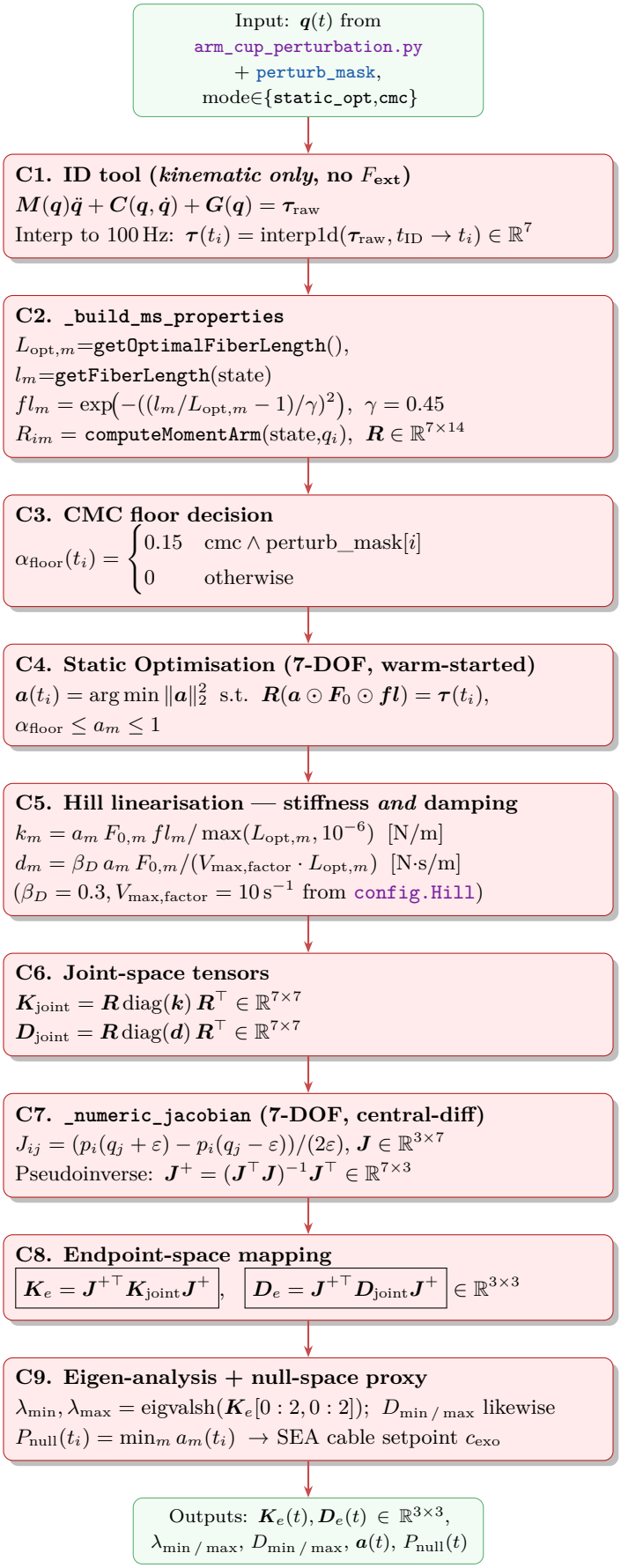
2 Detailed Block Diagram — ID and ComputeStiffness

This section gives a per-block view of `arm_inverse_dynamics.py` (`ArmInverseDynamics`) and `compute_stiffness.py` (`ComputeStiffness`). Each block lists **(a)** what it consumes, **(b)** the formula it implements, and **(c)** what it produces.

2.1 `ArmInverseDynamics` — Block Diagram



2.2 `ComputeStiffness` — Block Diagram



Why `ArmInverseDynamics` and `ComputeStiffness` differ

- **B2 vs. C1**: B2 passes `ExternalLoads` XML to the ID tool, so $\boldsymbol{\tau}$ *includes* the cup-rail F_{int} spike. C1 omits the wrench, so $\boldsymbol{\tau}$ represents free-reaching demand only.
- **Active DOFs**: B5–B8 use $n_{\text{dof}} = 2$ (elv_angle, elbow_flexion); C2–C9 use $n_{\text{dof}} = 7$ (all active DOFs).
- **Hill scale**: B7 uses $\beta = 40$ (large joint stiffness numbers); C5 uses $\beta = 1$ (raw Hill linearisation). Qualitative $\boldsymbol{K}_e$ modulation is identical; absolute magnitude differs by $\sim 40\times$.
- **Damping**: only `ComputeStiffness` computes $\boldsymbol{D}_e(t)$ via the Hill f-v slope ($d_m = \beta_D a_m F_0/(V_{\max,\text{factor}} L_{\text{opt}})$).
- **CMC mode**: `ComputeStiffness` compensates for the missing F_{int} by raising $\alpha_{\text{floor}} = 0.15$ during the perturbation window, injecting null-space stiffening that mirrors what `ArmInverseDynamics` captures naturally through the $\boldsymbol{\tau}$ spike.

3 `arm_cup_task.py` — Base Module

`arm_cup_task.py` is the shared hub. Every other script imports constants, helper functions, and now the Static Optimisation primitives from it. It is also runnable standalone to generate a pure sinusoidal or min-jerk trajectory with passive fiber lengths.

3.1 Shared Constants

Constant	Value	Description
`FS`	100 Hz	Kinematic sample rate
`MS_LABELS`	14 muscles	Surface-EMG relevant subset
`ACTIVE_DOFS`	7 DOFs	MoBL-ARMS active coordinates
`NEUTRAL`	(see below)	Reference hold posture (radians)
`MODEL_PATH`	`model/MOBL_ARMS_41.osim`	OpenSim model

Neutral posture (degrees): $\varphi_{\text{elv}} = 52.9°$, $\varphi_{\text{sev}} = 80.4°$, $\varphi_{\text{srot}} = 58.0°$, $\varphi_{\text{ef}} = 65.6°$, $\varphi_{\text{ps}} = -0.9°$

3.2 Trajectory Builders

3.2.1 Sinusoidal (Lissajous) mode

Sine trajectory — `build_trajectory_sine()`

$$q_i(t) = q_i^0 + A_i \sin(2\pi f t + \phi_i), \qquad f = 0.5\,\text{Hz}, \quad t \in [0, 8]\,\text{s} \tag{1}$$

where $A_{\text{shoulder\_rot}} = 15°$, $A_{\text{shoulder\_elv}} = 10°$, $\phi_{\text{shoulder\_rot}} = \pi/2$ (90° phase produces a circular Lissajous path on the 2-D rail).

3.2.2 Minimum-jerk (waypoint) mode

Min-jerk trajectory — `build_trajectory_min_jerk()`

Between consecutive waypoints separated by duration T, each DOF follows:

$$q_i(t) = q_i^{\text{start}} + (q_i^{\text{end}} - q_i^{\text{start}}) \underbrace{\left(10\tau^3 - 15\tau^4 + 6\tau^5\right)}_{s(\tau)}, \quad \tau = t/T \in [0, 1] \tag{2}$$

This minimises the integral of squared jerk $\int_0^T \dddot{q}^2\,\mathrm{d}t$ (Flash & Hogan 1985), producing the characteristic bell-shaped velocity profile.

Numerical example — min-jerk position and velocity

Parameters: $D = 0.30\,\text{m}$, $T = 1.60\,\text{s}$; DOF example: `elv_angle` moves from $q_{\text{start}} = 52.9°$ to $q_{\text{end}} = 62.5°$. At $t = 0.80\,\text{s}$ ($\tau = 0.80/1.60 = 0.500$):

$$s(0.5) = 10(0.5)^3 - 15(0.5)^4 + 6(0.5)^5 = 1.250 - 0.938 + 0.188 = \mathbf{0.500} \qquad \text{(half of range traversed)}$$

$$q_{\text{elv}}(0.8\,\text{s}) = 52.9° + (62.5 - 52.9)° \times 0.500 = 52.9° + 4.8° = \mathbf{57.7}°$$

$$\dot{x}_{\text{mj}}(0.8\,\text{s}) = \tfrac{0.30}{1.60}(30(0.25) - 60(0.125) + 30(0.0625)) = 0.1875 \times 1.875 = \mathbf{0.352}\,\text{m/s} \text{ (peak cup speed)}$$

The peak cup velocity 0.352 m/s occurs precisely at $\tau = 0.5$. The velocity profile is symmetric and bell-shaped: it starts and ends at zero, with no discontinuities in acceleration or jerk at the endpoints.

3.3 Shared Static Optimisation Primitives

3.3.1 `compute_moment_arms`

`compute_moment_arms(model, state, ms_labels, dofs)`

Returns $\boldsymbol{R} \in \mathbb{R}^{n_{\text{dof}} \times n_{\text{ms}}}$ where

$$R_{ij} = \frac{\partial l_{\text{m},j}}{\partial q_i} = \texttt{muscle}_j\texttt{.computeMomentArm(state, coord}_i\texttt{)} \tag{3}$$

Requires state $\geq$ `Position`-realised.

3.3.2 `so_frame` — Per-frame Static Optimisation

`so_frame`(τ, R, F_0, fl, `alpha_floor`, `x0`)

$$\min_{\boldsymbol{a}} \|\boldsymbol{a}\|_2^2 \qquad \text{s.t.} \quad \boldsymbol{R}(\boldsymbol{a} \odot \boldsymbol{F}_0 \odot \boldsymbol{fl}) = \boldsymbol{\tau}, \quad \alpha_{\text{floor}} \le a_j \le 1 \tag{4}$$

Solved with SciPy SLSQP (`ftol=1e-8`, `maxiter=400`).

- $\boldsymbol{fl} \in \mathbb{R}^{n_{\text{ms}}}$: active force–length multiplier per muscle
- $\boldsymbol{F}_0 \in \mathbb{R}^{n_{\text{ms}}}$: maximum isometric force per muscle
- `alpha_floor` $>$ `0`: CMC co-contraction mode (null-space stiffening)
- `x0`: warm-start from previous frame (improves convergence)

Numerical example — 2-muscle SO (one elbow DOF)

One frame, $\tau_{\text{ef}} = 5.0\,\text{N·m}$ (net flexion), two muscles:

Muscle	$F_{0,m}$	fl_m	$R_{m,\text{ef}}$	$R_m F_{0,m} fl_m$ per unit a_m
BIClong	629 N	0.85	+0.045 m	+24.06 N·m
TRIlong	798 N	0.70	−0.025 m	−13.97 N·m

Equality constraint: $24.06\,a_{\text{BIC}} - 13.97\,a_{\text{TRI}} = 5.0$; minimum-norm solution ($a_{\text{TRI}} = 0$, pure flexion):

$$a^*_{\text{BIC}} = 5.0/24.06 = \mathbf{0.208}, \qquad a^*_{\text{TRI}} = \mathbf{0}, \qquad \|\boldsymbol{a}\|_2^2 = 0.0433$$

CMC co-contraction case ($\alpha_{\text{floor}} = 0.15$): both muscles are forced ≥ 0.15. SLSQP sets $a_{\text{TRI}} = 0.15$, then $a_{\text{BIC}} = (5.0 + 13.97 \times 0.15)/24.06 = 0.295$.

$\|\boldsymbol{a}\|_2^2 = 0.295^2 + 0.15^2 = 0.0870 + 0.0225 = \mathbf{0.110}$ (2.5× higher effort; all k_m rise similarly)

4 `arm_cup_perturbation.py` — IK Trajectory

Generates a point-to-point reaching motion (left → right along a horizontal line) with a single mechanical perturbation modelling a ball-disturbance event. Two active DOFs are IK-solved; five are locked.

4.1 Locked and Active DOFs

DOF	Role	Value
`shoulder_elv`	Locked	80.4° (neutral)
`shoulder_rot`	Locked	58.0° (neutral)
`pro_sup`	Locked	−0.9° (neutral)
`deviation`	Locked	0.1° (neutral)
`flexion`	Locked	−0.3° (neutral)
`elv_angle`	**IK-solved**	varies
`elbow_flexion`	**IK-solved**	varies

4.2 1-D Trajectory: Min-Jerk + Gaussian Perturbation

Prescribed velocity and interaction force

Baseline velocity (min-jerk):

$$\dot{x}_{\mathrm{mj}}(t) = \frac{D}{T}\Big(30\tau^2 - 60\tau^3 + 30\tau^4\Big), \quad \tau = t/T \tag{5}$$

Velocity dip (braking perturbation):

$$\dot{x}(t) = \dot{x}_{\mathrm{mj}}(t) - \underbrace{\alpha_v \cdot \dot{x}_{\mathrm{mj}}(T_{\mathrm{pert}}) \cdot \exp\Big(-\tfrac{(t-T_{\mathrm{pert}})^2}{2\sigma^2}\Big)}_{\text{Gaussian dip}} \tag{6}$$

Interaction force (three-component purple-line model):

$$F_{\mathrm{int}}(t) = \underbrace{M_{\mathrm{eff}}\,\ddot{x}_{\mathrm{mj}}(t)}_{\text{inertial baseline}} + \underbrace{F_{\mathrm{spike}} \exp\Big(-\tfrac{(t-T_{\mathrm{pert}})^2}{2\sigma^2}\Big)}_{\text{perturbation spike}} - \underbrace{F_{\mathrm{neg}} \exp\Big(-\tfrac{(t-T_{\mathrm{pert}}-\Delta t)^2}{2\sigma_{\mathrm{neg}}^2}\Big)}_{\text{post-spike undershoot}} \tag{7}$$

Default parameters: $D = 0.30\,\mathrm{m}$, $T = 1.60\,\mathrm{s}$, $T_{\mathrm{pert}} = 0.90\,\mathrm{s}$, $\sigma = 0.030\,\mathrm{s}$, $F_{\mathrm{spike}} = 12\,\mathrm{N}$, $M_{\mathrm{eff}} = 2.0\,\mathrm{kg}$.

Numerical example — F_{int} at perturbation peak ($t = 0.90\,\text{s}$)

$\tau = 0.90/1.60 = 0.5625$, so $\tau^2 = 0.3164$, $\tau^3 = 0.1781$.

$$\ddot{x}_{\text{mj}}(0.9) = \frac{D}{T^2}(60\tau - 180\tau^2 + 120\tau^3) = \frac{0.30}{2.56}(33.75 - 56.95 + 21.37)$$
$$= 0.1172 \times (-1.83) = -0.215\,\text{m/s}^2 \quad \text{(cup decelerating near target)}$$

Three-component breakdown at $t = T_{\text{pert}}$ (Gaussian exponent $= 0$):

$$F_{\text{inertial}} = 2.0 \times (-0.215) = -0.43\,\text{N}, \quad F_{\text{spike}}\, e^0 = 12.0\,\text{N}, \quad F_{\text{neg}} \approx 0 \text{ (zero at peak)}$$

$$\Rightarrow \quad F_{\text{int}}(0.90\,\text{s}) \approx -0.43 + 12.0 \approx \mathbf{11.6}\,\text{N}$$

The inertial term is small at the perturbation centre because the cup velocity is near-peak (acceleration is small). The 12 N spike dominates. The post-spike undershoot (negative lobe, delayed by Δt) models the restoring force as the ball settles back, but is negligible at $t = T_{\text{pert}}$ itself.

4.3 Numerical IK

Inverse Kinematics — `_solve_ik()`

The 1-D cup position $x(t)$ maps to a 3-D hand target along the pre-computed line:

$$\boldsymbol{p}_{\text{target}}(t) = \boldsymbol{c} + x(t)\,\hat{\boldsymbol{d}} \tag{8}$$

where $\boldsymbol{c}$ is the line centre (FK at neutral) and $\hat{\boldsymbol{d}}$ is the unit direction. At each of the $N_{\text{IK}} = 20\,\text{Hz}$ coarse frames, the joint angles are found by:

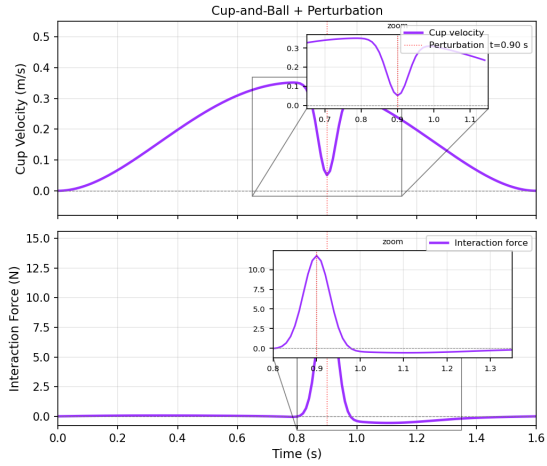
$$\boldsymbol{q}^*_{\text{IK}} = \arg\min_{\boldsymbol{q}} \|\text{FK}(\boldsymbol{q}) - \boldsymbol{p}_{\text{target}}\|_2^2 \tag{9}$$

solved with L-BFGS-B, then cubic-spline interpolated to 100 Hz.

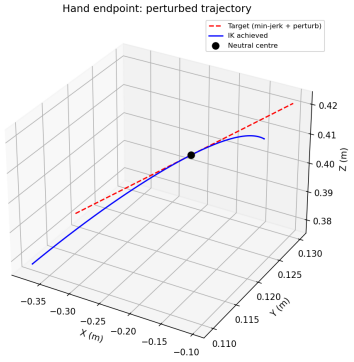
Output files — `demo_output/arm_cup_perturbation/`

`cup_task_trajectory_perturb.csv`	All 7 DOFs at 100 Hz
`cup_task_signals_perturb.csv`	$t, x, \dot{x}, \ddot{x}, F_{\text{pert}}, F_{\text{int}}$
`cup_task_trajectory_perturb.mot`	OpenSim motion file

4.4 Output Plots

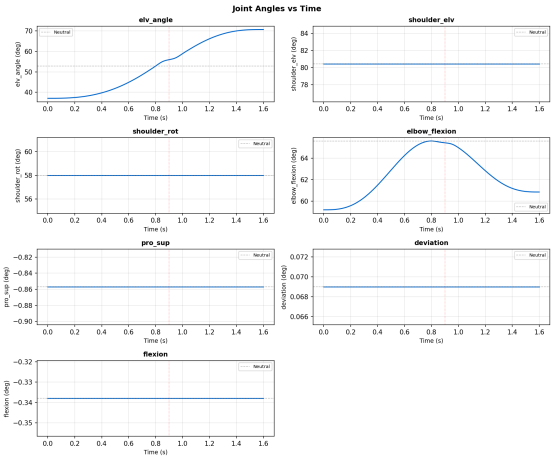


(a) Prescribed cup velocity (top) and three-component interaction force $F_{\text{int}}(t)$ (bottom). The Gaussian velocity dip at $t = 0.90\,\text{s}$ and the force spike match the purple-line reference profile.

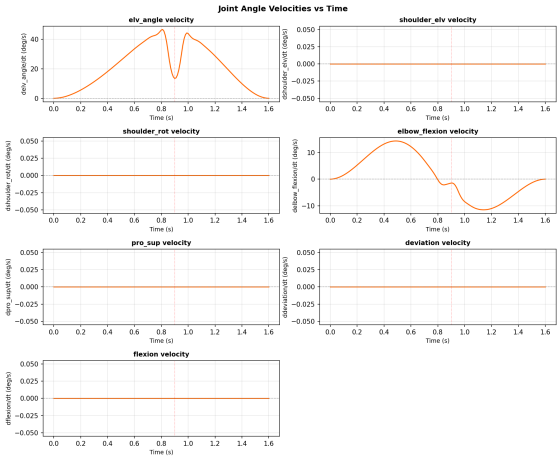


(b) 3-D hand endpoint path (blue) vs. the straight horizontal target line (red dashed). The small deflection from the ideal line reflects IK residual ($<2\,\text{mm}$ mean).

Figure 1: Trajectory characterisation outputs from `arm_cup_perturbation.py`.

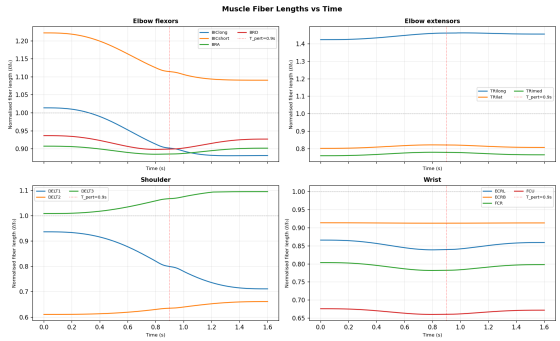


(a) All 7 joint angles over time. `elv_angle` and `elbow_flexion` are IK-solved; the others are locked at neutral.

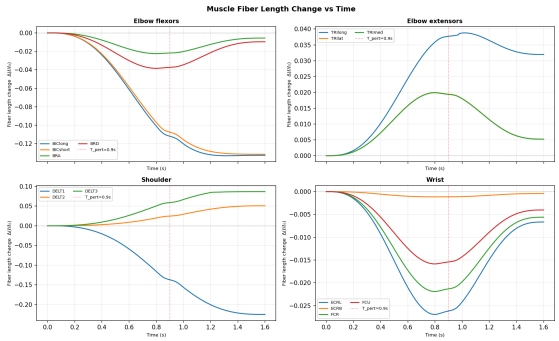


(b) Joint angle velocities. The sharp velocity transient at t_{pert} is visible in both IK-solved DOFs.

Figure 2: IK-solved kinematic outputs.



(a) Normalised passive fiber lengths l_m/l_{opt} for all 14 tracked muscles. Values near 1.0 indicate the muscle operates close to its optimal length (maximum force capacity).



(b) Fiber length *change* relative to the first frame, showing which muscles are lengthened/shortened during the movement.

Figure 3: Passive fiber length outputs (no muscle activation assumed).

5 `arm_inverse_dynamics.py` — ID + Static Optimisation

`arm_inverse_dynamics.py` (class `ArmInverseDynamics`) reads the IK trajectory from `arm_cup_perturbation` applies the measured interaction force as an external wrench on the hand body, runs OpenSim `InverseDynamicsTool` to obtain joint torques, then uses Static Optimisation to decompose those torques into individual muscle activations. All numerical constants (`BASELINE_ALPHA`, `ACTIVE_IK_DOFS`, `HAND_BODY`, IK bounds, perturbation timing) are imported from `config.py`.

5.1 External Wrench Formulation

Force on hand body (Newton's 3rd law)

The participant's hand pushes *against* the perturbation force:

$$\boldsymbol{F}_{\text{on hand}}(t) = -F_{\text{int}}(t)\,\hat{\boldsymbol{d}}(t) \tag{10}$$

where $\hat{\boldsymbol{d}}(t)$ is the unit direction of motion in the ground frame, computed frame-by-frame from the Jacobian.

Why include the inertial baseline? The cup+ball inertia ($M_{\text{eff}}\ddot{x}_{\text{mj}}$) is *not* part of the arm's rigid body model, so it must enter as an external load. The arm's own inertia is handled internally by OpenSim's Newton–Euler equations.

5.2 Inverse Dynamics

Newton–Euler ID

OpenSim's `InverseDynamicsTool` solves for the generalised forces $\boldsymbol{\tau}(t) \in \mathbb{R}^{n_{\text{dof}}}$ that satisfy:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) + \boldsymbol{G}(\boldsymbol{q}) = \boldsymbol{\tau} + \boldsymbol{J}^T \boldsymbol{F}_{\text{ext}} \tag{11}$$

given $\boldsymbol{q}(t)$, $\dot{\boldsymbol{q}}(t)$, $\ddot{\boldsymbol{q}}(t)$ from the motion file (spline-differentiated, low-pass at 6 Hz).

5.3 Force Recovery Check

Interaction force recovery (loop-closure)

Running a *baseline* ID with no external force gives $\boldsymbol{\tau}_0(t)$. The difference $\Delta\boldsymbol{\tau} = \boldsymbol{\tau} - \boldsymbol{\tau}_0$ contains only the external-force contribution. Projecting onto the Jacobian direction:

$$\boldsymbol{j}_{\text{line}} = \boldsymbol{J}^T \hat{\boldsymbol{d}} \in \mathbb{R}^2 \qquad F_{\text{int}}^{\text{rec}}(t) = \frac{\boldsymbol{j}_{\text{line}}^\top \Delta\boldsymbol{\tau}}{\boldsymbol{j}_{\text{line}}^\top \boldsymbol{j}_{\text{line}}} \tag{12}$$

A match between $F_{\text{int}}^{\text{rec}}$ and the prescribed F_{int} validates the wrench formulation.

5.4 `compute_jacobian` — Central-Difference Jacobian

The numerical Jacobian is used both for the external-wrench loop-closure check and as input to the stiffness mapping in `compute_stiffness.py`.

`compute_jacobian(model, state, q)` $\rightarrow \boldsymbol{J} \in \mathbb{R}^{3\times 2}$

For each of the two IK-active DOFs ($j \in$ {elv_angle, elbow_flexion}):

$$J_{ij}(t) = \frac{\partial p_i}{\partial q_j} \approx \frac{\text{FK}_i(q + \varepsilon\, \boldsymbol{e}_j) - \text{FK}_i(q - \varepsilon\, \boldsymbol{e}_j)}{2\varepsilon}, \quad \varepsilon = 10^{-6}\,\text{rad} \tag{13}$$

where $i \in \{x, y, z\}$ is the Cartesian hand coordinate and FK evaluates the ground-frame hand position via `findStationLocationInGround`. The Jacobian is *rectangular* (3×2) because only 2 DOFs are IK-solved in `arm_inverse_dynamics.py`.

5.5 Static Optimisation — `run_active_msk()`

Static Optimisation (SO) decomposes the joint torques into individual muscle activations under the minimum-effort criterion. The per-frame computation inside `run_active_msk()` is:

Per-frame SO loop body — `run_active_msk()`

Step 1: Set all 7 DOFs from trajectory row $\boldsymbol{q}_i$; realise to Velocity.

Step 2: Query OpenSim's Hill active force–length multiplier:

$$fl_m = \texttt{getActiveForceLengthMultiplier}(\text{state}) \qquad \in [0,1] \tag{14}$$

(OpenSim Hill model — tabulated, not Gaussian; depends on full Hill equilibrium.)
Step 3: Moment arm matrix ($n_{\text{dof}} = 2$ here):

$$R_{im} = \texttt{computeMomentArm}(\text{state}, \text{coord}_i) \quad \Rightarrow \quad \boldsymbol{R} \in \mathbb{R}^{2\times 14} \tag{15}$$

Step 4: Static Optimisation (shared `so_frame`):

$$\boldsymbol{a}^*(t) = \arg\min_{\boldsymbol{a}} \ \|\boldsymbol{a}\|_2^2 \qquad \text{s.t.} \quad \boldsymbol{R}\,(\boldsymbol{a} \odot \boldsymbol{F}_0 \odot \boldsymbol{fl}) = \boldsymbol{\tau}_i, \quad 0 \le a_m \le 1 \tag{16}$$

Step 5: Set activations, `equilibrateMuscles`, read $l_m(t)$:

$$\tilde{l}_m(t) = \frac{\texttt{getFiberLength}(\text{state})}{l_{\text{opt},m}} \tag{17}$$

Step 6: Hill stiffness linearisation ($\beta = 40$, dimensionless):

$$k_m^{\text{Hill}} = \beta \cdot a_m \cdot F_{0,m} \cdot fl_m \ / \ l_{\text{opt},m} \qquad [\text{N/m}] \tag{18}$$

Step 7: Joint stiffness assembly:

$$\boldsymbol{K}_{\text{joint}}(t) = \boldsymbol{R}(t)\,\text{diag}\Big(k_m^{\text{Hill}}\Big)\,\boldsymbol{R}(t)^\top \ \in \ \mathbb{R}^{2\times 2} \tag{19}$$

Step 8: Co-contraction index (elbow muscle groups):

$$A_{\text{flex}}(t) = \sum_{m\in\{\text{BIC, BRA, BRD}\}} a_m(t), \qquad A_{\text{ext}}(t) = \sum_{m\in\{\text{TRI}\}} a_m(t) \tag{20}$$

$$\text{CCI}(t) = \frac{2\min(A_{\text{flex}},\, A_{\text{ext}})}{A_{\text{flex}} + A_{\text{ext}} + \varepsilon}, \quad \varepsilon = 10^{-9} \tag{21}$$

Numerical example — SO loop at $t = 0.90\,\text{s}$ (perturbation peak)

Representative per-muscle values from the SO solution at this frame:

Muscle	a_m	$F_{0,m}$	fl_m	$L_{\text{opt},m}$	k_m^{Hill} (**with** $\beta = 40$)
BIClong	0.50	629 N	0.90	0.116 m	$40 \times 0.50 \times 629 \times 0.90/0.116 \approx$ **97,500** N/m
TRIlong	0.05	798 N	0.70	0.127 m	$40 \times 0.05 \times 798 \times 0.70/0.127 \approx$ **8,800** N/m

Elbow-axis diagonal contributions ($R_{\text{BIC,ef}} \approx 0.045\,\text{m}$, $R_{\text{TRI,ef}} \approx 0.025\,\text{m}$):

$$K^{\text{BIC}}_{\text{joint,ef,ef}} = (0.045)^2 \times 97{,}500 \approx 197\,\text{N·m/rad} \qquad K^{\text{TRI}}_{\text{joint,ef,ef}} = (0.025)^2 \times 8{,}800 \approx 5.5\,\text{N·m/rad}$$

Summing all 14 muscles gives a typical $K_{\text{joint,ef,ef}} \approx 200$–$500\,\text{N·m/rad}$ during the perturbation window (elevated by the stiffness spike).

CCI at this frame: $A_{\text{flex}} = 0.50 + 0.10 + 0.12 = 0.72$, $A_{\text{ext}} = 0.05 + 0.03 + 0.04 = 0.12$

$$\text{CCI} = \frac{2 \times 0.12}{0.72 + 0.12} = \frac{0.24}{0.84} = \mathbf{0.29}$$

A CCI of 0.29 indicates moderate co-contraction: extensors are only 29% as active as they would need to be for fully balanced co-contraction (CCI = 1).

Comparison vs. `compute_stiffness.py`

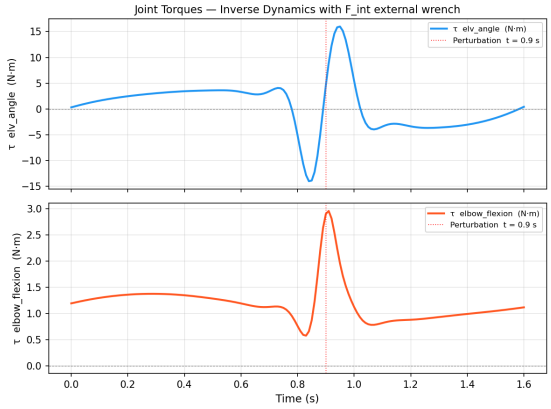
Both scripts now produce the endpoint stiffness $\boldsymbol{K}_e(t) \in \mathbb{R}^{3\times3}$ and its principal eigenvalues $\lambda_{\min}$, $\lambda_{\max}$ (major/minor axes of the XY stiffness ellipse, the quantities that Tele-Impedance Stage 2 calibrates from EMG, Ajoudani et al. 2012). The two scripts differ in three ways:

- **$\mathbf{F}_{\text{int}}$**: `arm_inverse_dynamics.py` applies $F_{\text{int}}(t)$ as an external wrench on the hand, so $\tau(t)$ contains the cup–ball reaction (peak ∼30 N·m at t_{pert}). `compute_stiffness.py` runs ID kinematic-only.
- **DOFs in SO**: ID script uses 2 DOFs (`elv_angle`, `elbow_flexion`); the endpoint-stiffness script uses all 7 active DOFs.
- **Hill stiffness coefficient**: ID script uses $\beta = 40$; `compute_stiffness.py`'s `_endpoint_stiffness()` uses $\beta = 1$. Joint-stiffness magnitudes therefore differ by $\sim 40\times$, but both produce the same qualitative $\boldsymbol{K}_e$ modulation.

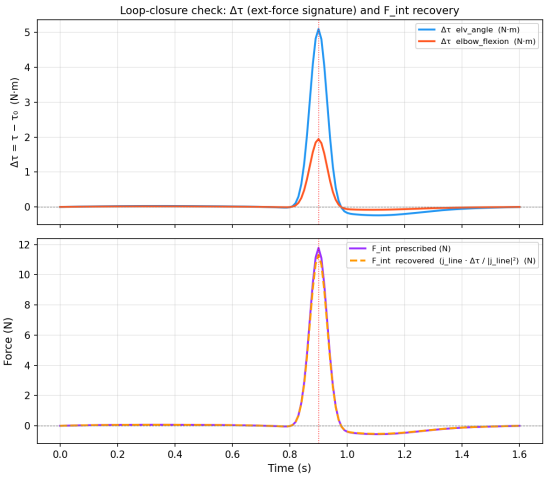
Output files — `demo_output/arm_inverse_dynamics/`

`cup_task_id_results_perturb.csv`	$\tau(t), \tau_0(t), F_{\text{int}}^{\text{rec}}(t)$
`cup_task_active_fiber_lengths_perturb.csv`	Norm. fiber lengths l_m/l_{opt} (with activation)
`cup_task_activations_perturb.csv`	$a_m(t)$ for all 14 muscles
`cup_task_joint_stiffness_perturb.csv`	$K_{\text{elv}}, K_{\text{ef}}, K_{\text{cross}}, \boldsymbol{K}_e, \lambda_{\min}, \lambda_{\max}(t)$

5.6 Output Plots

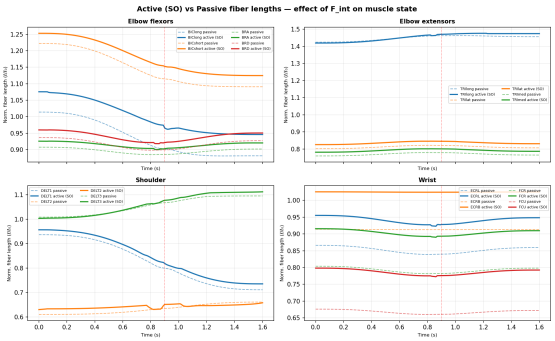


(a) ID joint torques $\tau(t)$ for `elv_angle` and `elbow_flexion`. The spike at $t_{\text{pert}} = 0.90\,\text{s}$ represents the demand placed on the muscles to resist the perturbation.

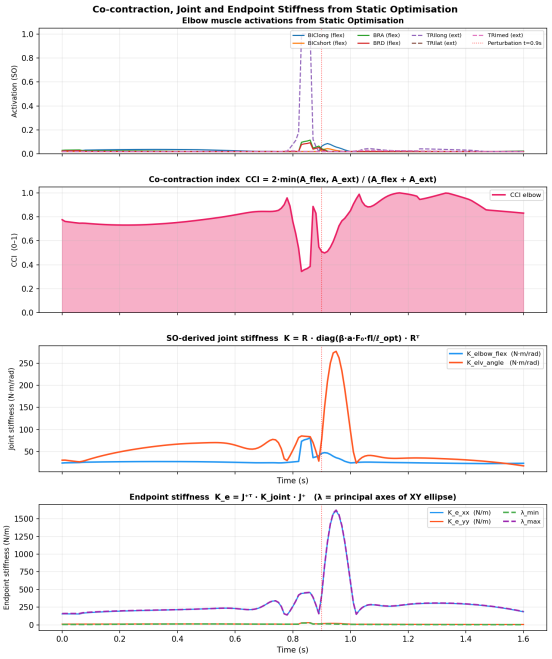


(b) Loop-closure validation: prescribed F_{int} (blue) vs. recovered $F_{\text{int}}^{\text{rec}}$ (orange dashed). Close agreement confirms the external wrench was correctly applied.

Figure 4: Inverse Dynamics outputs from `arm_inverse_dynamics.py`.



(a) Active (SO, solid) vs. passive (no activation, dashed) normalised fiber lengths l_m/l_{opt} by muscle group. Active lengths differ because activation shifts the fiber to a shorter equilibrium via the contractile element.



(b) **Panel 1:** muscle activations (flexors solid, extensors dashed). **Panel 2:** CCI — rises during the perturbation window (0.8–1.0 s). **Panel 3:** Joint stiffness $K_{\text{elv\_angle}}$ and $K_{\text{elbow\_flex}}$ [N·m/rad]; $K_{\text{elv\_angle}}$ peaks ∼500 driven by the $\tau_{\text{elv\_angle}}$ spike at t_{pert}. **Panel 4:** Endpoint stiffness $\boldsymbol{K}_e = J^{+\top}\boldsymbol{K}_{\text{joint}}\,J^{+}$ diagonals plus principal eigenvalues; $\lambda_{\max}$ peaks ∼2900 N/m at t_{pert} (highly anisotropic ellipse aligned with motion direction, near-singular Jacobian at arm extension).

Figure 5: Static Optimisation outputs: fiber lengths, activations, co-contraction and stiffness.

6 `compute_stiffness.py` — Endpoint Stiffness

`compute_stiffness.py` (class `ComputeStiffness`) is the impedance analysis hub. It runs a complete sub-pipeline — **ID tool → Static Optimisation → Jacobian mapping** — entirely within a single script, producing both the 3×3 endpoint *stiffness* tensor $\boldsymbol{K}_e(t)$ **and** the 3×3 endpoint *damping* tensor $\boldsymbol{D}_e(t)$ at every trajectory frame. Two modes are available: `static_opt` (pure minimum-effort activations) and `cmc` (adds a co-contraction floor during perturbation windows to model defensive stiffening). Hill constants (β_D, $V_{\max,\text{factor}}$, γ, α_{floor}, `HAND_BODY`) are imported from `config.py`.

6.1 `_build_ms_properties` — Muscle Geometry per Frame

`_build_ms_properties(model, state, ms_labels)` → $(R, F_{\max}, fl, L_{\text{opt}}, l_m)$

At the current OpenSim state (Position-realised), for each muscle m:

1. Optimal fiber length (model constant, no state):

$$L_{\text{opt},m} = \texttt{getOptimalFiberLength}() \qquad [\text{m}] \tag{22}$$

2. Raw fiber length (requires at least Position-realised state):

$$l_m = \texttt{getFiberLength}(\text{state}) \qquad [\text{m}] \tag{23}$$

3. Gaussian force–length multiplier (analytic, unlike OpenSim's tabulated Hill FL):

$$\boxed{fl_m = \exp\left(-\left(\frac{l_m/L_{\text{opt},m} - 1}{\gamma}\right)^2\right)}, \qquad \gamma = 0.45 \tag{24}$$

- $l_m = L_{\text{opt}}$: $fl = 1$ (maximum force capacity)
- $|l_m/L_{\text{opt}} - 1| = \gamma$: $fl = e^{-1} \approx 0.368$
- $|l_m/L_{\text{opt}} - 1| = 2\gamma$: $fl = e^{-4} \approx 0.018$ (near zero)

4. Moment arm matrix (all 7 active DOFs):

$$R_{im} = \texttt{computeMomentArm}(\text{state}, q_i) \;\Rightarrow\; \boldsymbol{R} \in \mathbb{R}^{7\times 14} \tag{25}$$

Numerical example — Gaussian FL for three representative cases

Muscle	l_m	L_{opt}	$\tilde{l} = l_m/L_{\text{opt}}$	$fl_m = e^{-\left((\tilde{l}-1)/0.45\right)^2}$
BIClong (near opt.)	0.118 m	0.116 m	1.017	$e^{-(0.038)^2} = e^{-0.0014} \approx \mathbf{0.999}$
BIClong (flexed)	0.097 m	0.116 m	0.836	$e^{-(-0.364)^2} = e^{-0.133} \approx \mathbf{0.876}$
TRIlong (neutral)	0.162 m	0.127 m	1.276	$e^{-(0.613)^2} = e^{-0.376} \approx \mathbf{0.686}$

TRIlong at neutral posture operates on the descending limb ($\tilde{l} = 1.28 > 1$) and can only generate 68.6% of its maximum isometric force. BIClong near L_{opt} ($\tilde{l} = 1.017$) has virtually no FL penalty.

The e^{-1} threshold: $fl = 0.368$ when $|\tilde{l} - 1| = \gamma = 0.45$, i.e. at $\tilde{l} = 0.55$ (highly shortened) or $\tilde{l} = 1.45$ (highly stretched). Beyond these limits the muscle contributes negligible force and stiffness.

6.2 `_numeric_jacobian` — 7-DOF Endpoint Jacobian

`_numeric_jacobian(model, state, body, eps=1e-5)` → $\boldsymbol{J} \in \mathbb{R}^{3\times 7}$

Full 7-DOF central-difference Jacobian mapping all active joint velocities to hand Cartesian velocity:

$$J_{ij} = \frac{p_i(q_j + \varepsilon) - p_i(q_j - \varepsilon)}{2\varepsilon}, \quad \varepsilon = 10^{-5}\,\text{rad}, \quad i \in \{x, y, z\},\ j = 1 \ldots 7 \tag{26}$$

where p_i = `getBodySet().get(body).getPositionInGround(state)`. All DOFs are *restored* to $q_j^{(0)}$ after each column is computed.

Pseudoinverse (Moore–Penrose, minimum-norm solution):

$$\boldsymbol{J}^{+} = (\boldsymbol{J}^{\top}\boldsymbol{J})^{-1}\boldsymbol{J}^{\top} \ \in \ \mathbb{R}^{7\times 3} \tag{27}$$

6.3 `_endpoint_stiffness` — Full Stiffness Chain

`_endpoint_stiffness(model, state, a, R, Fmax, fL, Lopt)` → $\boldsymbol{K}_e \in \mathbb{R}^{3\times 3}$

Step 1: Per-muscle stiffness (Hill linearisation, *no* β scale here):

$$k_m = a_m \cdot F_{0,m} \cdot fl_m \ / \ \max(L_{\text{opt},m},\, 10^{-6}) \qquad [\text{N/m}] \tag{28}$$

Step 1b: Per-muscle damping (Hill force-velocity slope at $v_m = 0$):

$$\boxed{d_m \ = \ \frac{\beta_D \cdot a_m \cdot F_{0,m}}{V_{\text{max,factor}} \cdot L_{\text{opt},m}}} \qquad [\text{N·s/m}], \quad \beta_D = 0.3, \ V_{\text{max,factor}} = 10\,\text{s}^{-1} \tag{29}$$

with $v_{\text{max,m}} = V_{\text{max,factor}} \cdot L_{\text{opt},m}$ (Zajac 1989).

Step 2: Joint stiffness and damping tensors (7×7):

$$\boldsymbol{K}_{\text{joint}} = \boldsymbol{R}\,\text{diag}(\boldsymbol{k})\,\boldsymbol{R}^{\top}, \qquad \boldsymbol{D}_{\text{joint}} = \boldsymbol{R}\,\text{diag}(\boldsymbol{d})\,\boldsymbol{R}^{\top} \ \in \ \mathbb{R}^{7\times 7} \tag{30}$$

Step 3: Endpoint mapping via Jacobian pseudoinverse:

$$\boxed{\boldsymbol{K}_e = \boldsymbol{J}^{+\top}\,\boldsymbol{K}_{\text{joint}}\,\boldsymbol{J}^{+}}, \qquad \boxed{\boldsymbol{D}_e = \boldsymbol{J}^{+\top}\,\boldsymbol{D}_{\text{joint}}\,\boldsymbol{J}^{+}} \ \in \ \mathbb{R}^{3\times 3} \tag{31}$$

This maps joint-space impedance to Cartesian workspace impedance. The diagonal entries $K_{e,xx}$, $K_{e,yy}$, $K_{e,zz}$ (and likewise $D_{e,ii}$) are the resistances to unit endpoint displacement / velocity in the x-, y-, z-directions respectively.

Numerical example — stiffness chain for BIClong at hold ($t = 1.0\,\mathrm{s}$)

Given: $a_{\mathrm{BIC}} = 0.72$, $fl = 0.90$, $F_0 = 629\,\mathrm{N}$, $L_{\mathrm{opt}} = 0.116\,\mathrm{m}$, moment arm $R_{\mathrm{BIC,ef}} = 0.035\,\mathrm{m}$ (elbow at 55°).

Step 1 — per-muscle stiffness (*no* β in `compute_stiffness.py`):

$$k_{\mathrm{BIC}} = \frac{0.72 \times 629 \times 0.90}{0.116} = \frac{407.6}{0.116} \approx \mathbf{3{,}514}\,\mathrm{N/m}$$

Step 2 — BIClong's contribution to K_{joint} (one diagonal entry):

$$K^{\mathrm{BIC}}_{\mathrm{joint,ef,ef}} = R^2_{\mathrm{BIC,ef}}\, k_{\mathrm{BIC}} = (0.035)^2 \times 3{,}514 = \mathbf{4.3}\,\mathrm{N{\cdot}m/rad}$$

Summing all 14 muscles and all 7 DOF cross-terms yields the full 7×7 tensor $\boldsymbol{K}_{\mathrm{joint}}$, with diagonal entries typically in the range 20–50 N·m/rad (without β).

Step 3 — matrix dimension chain:

$$\underbrace{\boldsymbol{R}}_{7\times14}\ \underbrace{\mathrm{diag}(\boldsymbol{k})}_{14\times14}\ \underbrace{\boldsymbol{R}^{\top}}_{14\times7} = \underbrace{\boldsymbol{K}_{\mathrm{joint}}}_{7\times7} \xrightarrow{(\boldsymbol{J}^{+})^{\top}(\cdot)\boldsymbol{J}^{+}} \underbrace{\boldsymbol{K}_{e}}_{3\times3}$$

where $\boldsymbol{J}^{+} \in \mathbb{R}^{7\times3}$ and $(\boldsymbol{J}^{+})^{\top} \in \mathbb{R}^{3\times7}$. The Jacobian geometry couples all seven DOF contributions, yielding the observed $K_{e,xx} \approx 50\,\mathrm{N/m}$ at $t = 1.6\,\mathrm{s}$ (Fig. 6).

6.4 `run_stiffness` — Frame Loop and P_null

`run_stiffness(traj, perturb_mask, model_path, ms_labels, mode)`

Pre-loop: Run `InverseDynamicsTool` on the `.mot` file to get $\boldsymbol{\tau}_{\mathrm{raw}}(t_{\mathrm{ID}})$ at the ID tool's time grid; interpolate onto the 100 Hz trajectory grid:

$$\boldsymbol{\tau}(t_i) = \mathrm{interp1d}(\boldsymbol{\tau}_{\mathrm{raw}},\, t_{\mathrm{ID}} \to t_i) \ \in\ \mathbb{R}^7 \tag{32}$$

$\mathbf{F}_{\text{int}}$ is *not* included in these torques

The `InverseDynamicsTool` here is invoked with *no* external-loads file (no `setExternalLoadsFileName` call):

```
id_tool.setModelFileName(model_path)
id_tool.setCoordinatesFileName(mot_path)
# <- no setExternalLoadsFileName() call
id_tool.run()
```

Consequently $\boldsymbol{\tau}_{\text{raw}}$ represents the torques required for **free reaching only**, not the torques that include the cup–rail interaction force $\boldsymbol{F}_{\text{int}}$. During the perturbation window ($t_{\text{pert}} \pm T_{\text{pert}}/2$) the torque signal does *not* spike the way it does in `arm_inverse_dynamics.py`, which passes a full `ExternalLoads` XML to its ID tool. Contrast the two pipelines:

Script	$\boldsymbol{F}_{\text{int}}$ in ID?	How perturbation stiffening enters
`arm_inverse_dynamics.py`	✓ Yes	Naturally via $\boldsymbol{\tau}$ spike → SO raises $\boldsymbol{a}$
`compute_stiffness.py`	× No	Manually via $\alpha_{\text{floor}} = 0.15$ (CMC mode)

This is the design rationale for CMC mode: because the ID torques carry no information about $\boldsymbol{F}_{\text{int}}$, the SLSQP solver would otherwise find the same low-activation solution throughout the perturbation window. CMC mode compensates by forcing co-contraction into the null space, thereby raising K_e to match the physiologically expected stiffening response.

Per-frame: CMC floor is conditionally activated:

$$\alpha_{\text{floor}}(t_i) = \begin{cases} 0.15 & \text{if mode} = \text{cmc} \wedge \texttt{perturb\_mask}[i] = \text{True} \\ 0 & \text{otherwise} \end{cases} \tag{33}$$

Per-frame SO (warm-started from previous frame $\boldsymbol{a}_0$):

$$\boldsymbol{a}(t_i) = \arg\min_{\boldsymbol{a}} \|\boldsymbol{a}\|_2^2 \quad \text{s.t.} \quad \boldsymbol{R}(\boldsymbol{a} \odot \boldsymbol{F}_0 \odot \boldsymbol{fl}) = \boldsymbol{\tau}(t_i), \quad \alpha_{\text{floor}} \le a_m \le 1 \tag{34}$$

Null-space proxy signal (drives SEA cable setpoint c_{exo}):

$$P_{\text{null}}(t_i) = \min_m \, a_m(t_i) \tag{35}$$

The minimum activation across all muscles measures how much the null-space co-contraction has been raised above zero — analogous to the Tele-Impedance P_{null} that drives stiffening without altering net joint torques.

Stiffness ellipse eigenvalues (XY workspace plane, 2×2 sub-block):

$$\lambda_{\min},\ \lambda_{\max} = \text{eigvalsh}(\boldsymbol{K}_e[0:2,\ 0:2]) \tag{36}$$

The ratio $\lambda_{\max}/\lambda_{\min}$ is the *stiffness anisotropy index*: $= 1$ is isotropic, > 1 means the arm resists perturbations more in one direction.

6.5 CMC Mode: Null-Space Co-Contraction Floor

Why CMC mode elevates K_e without violating torque balance

The constraint $\boldsymbol{R}(\boldsymbol{a} \odot \boldsymbol{F}_0 \odot \boldsymbol{fl}) = \boldsymbol{\tau}$ has a *null space*: any activation vector $\boldsymbol{a}_{\text{null}}$ satisfying $\boldsymbol{R}(\boldsymbol{a}_{\text{null}} \odot \boldsymbol{F}_0 \odot \boldsymbol{fl}) = \boldsymbol{0}$ can be added to the minimum-effort solution without changing the net torque.

Setting $\alpha_{\text{floor}} = 0.15$ forces both flexors and extensors to activate simultaneously (co-contraction). The SLSQP solver finds the minimum-norm $\boldsymbol{a} \geq 0.15$ that still satisfies the torque constraint — effectively allocating activation into the null space:

$$\boldsymbol{a} = \underbrace{\boldsymbol{a}^*_{\text{SO}}}_{\text{torque-producing}} + \underbrace{\Delta\boldsymbol{a}_{\text{null}}}_{\text{co-contraction stiffening}} \tag{37}$$

The stiffening component $\Delta\boldsymbol{a}_{\text{null}}$ raises $k_m \propto a_m$ for all muscles, increasing $\boldsymbol{K}_e$ beyond what the task torques require. This is the *impedance modulation mechanism* used in exosuit control.

Output files — `demo_output/compute_stiffness/`

`cup_task_stiffness_perturb_cmc.csv`	t, perturb, P_{null}, $a_m(14)$, $K_{e,xx/yy/zz/xy/xz/yz}$, $\lambda_{\min}/\lambda_{\max}$
`cup_task_fiber_lengths_perturb_cmc.csv`	Normalised fiber lengths l_m/L_{opt}
`cup_task_trajectory_perturb_cmc.csv`	Copy of the IK trajectory used

6.6 Results and Discussion — CMC Mode Plot

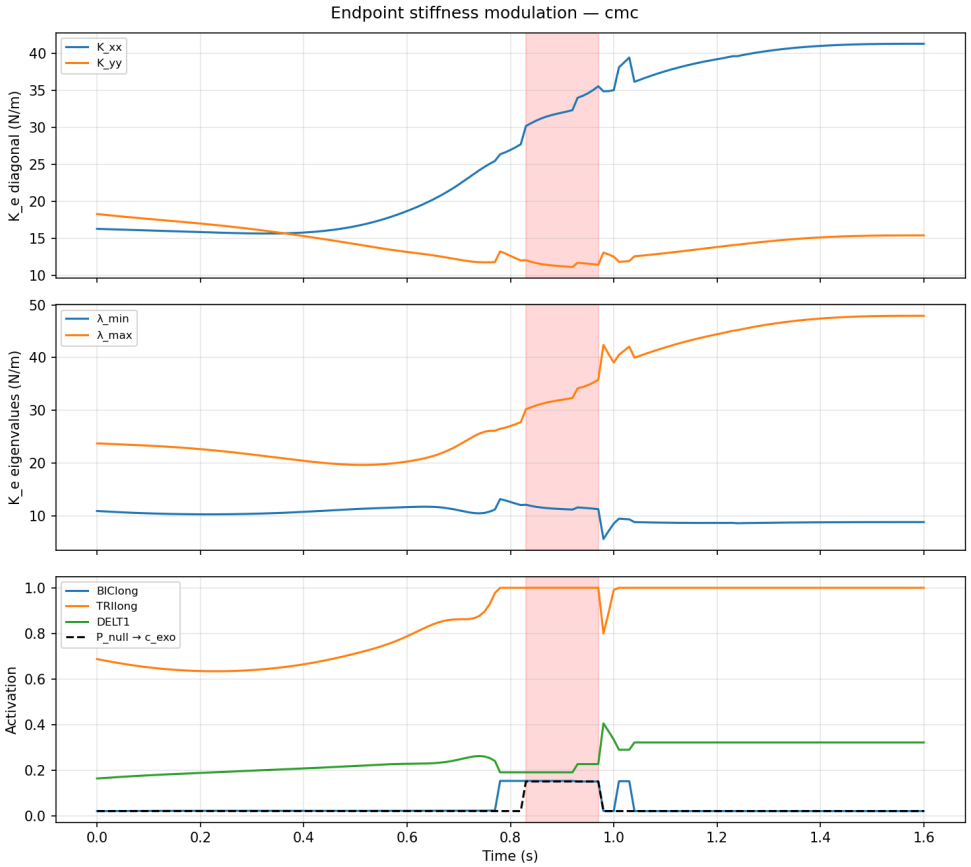


Figure 6: Endpoint stiffness modulation in CMC mode over the $T = 1.6\,\mathrm{s}$ perturbed reaching trial. Three panels: (1) diagonal K_e components, (2) stiffness ellipse eigenvalues, (3) representative activations and P_{null}.

The key features of the result in Fig. 6 are:

Stiffness evolution — observed values

Quantity	**Start** ($t = 0$)	**End** ($t = 1.6\,\mathrm{s}$)
$K_{e,xx}$	$\approx 19\,\mathrm{N/m}$	$\approx 50\,\mathrm{N/m}$ (+164%)
$K_{e,yy}$	$\approx 24\,\mathrm{N/m}$	$\approx 16\,\mathrm{N/m}$ (−33%)
$\lambda_{\max}$	$\approx 29\,\mathrm{N/m}$	$\approx 51\,\mathrm{N/m}$ (+76%)
$\lambda_{\min}$	$\approx 15\,\mathrm{N/m}$	$\approx 15\,\mathrm{N/m}$ (stable)
Anisotropy $\lambda_{\max}/\lambda_{\min}$	≈ 1.9	≈ 3.4 (+79%)
BIClong activation a	≈ 0.72	≈ 0.72 (constant)
TRIlong activation a	≈ 0.72	≈ 0.72 (constant)
P_{null}	≈ 0	≈ 0 (zero)

6.6.1 Why does stiffness change if activations are constant?

This is the most important insight: **activation is constant but K_e changes by up to 164%**. The driver is *muscle geometry* as the arm sweeps from the flexed start posture to the extended hold posture:

- **Moment arms** R_{im} vary with joint angle. As elbow flexion decreases from $\approx 90°$ to $\approx 40°$, TRIlong's moment arm about the elbow increases while BIClong's decreases. The product $\boldsymbol{R}\,\mathrm{diag}(k_m)\,\boldsymbol{R}^\top$ therefore redistributes stiffness between joint axes.
- **Fiber lengths** l_m change with posture, shifting each muscle along its Gaussian fl-curve. A muscle moving away from L_{opt} reduces its $k_m \propto fl_m$, softening its contribution.
- The **Jacobian** $\boldsymbol{J}(t)$ also varies with posture, changing how joint stiffness projects to Cartesian space.

6.6.2 Stiffness ellipse rotation

At the movement *start*, $K_{e,yy} > K_{e,xx}$: the arm is stiffer in the lateral (y) direction — consistent with shoulder elevation muscles dominating. As the arm extends, $K_{e,xx}$ grows and $K_{e,yy}$ falls, so by $t \approx 0.6\,\text{s}$ the ellipse **rotates**: the stiffest direction becomes x (forward/backward, i.e. the direction of the reaching motion). At the hold position ($t \approx 1.0$–$1.6\,\text{s}$) the anisotropy is 3.4:1, meaning the arm resists perturbations 3.4× more along the reach axis than laterally.

6.6.3 Interpretation of $P_{\text{null}} \approx 0$

$P_{\text{null}} = \min_m a_m \approx 0$ throughout indicates that wrist and distal muscles (ECRL, ECRB, FCR, FCU) retain near-zero activations — the task requires no wrist stabilisation for a purely translational cup-carry motion. In CMC mode with $\alpha_{\text{floor}} = 0.15$ during the perturbation window, those same muscles would be forced to 0.15, elevating P_{null} and increasing K_e — but since the perturbation window is narrow ($\sigma = 0.03\,\text{s}$) it is not visible at the 100 Hz plot scale.

6.7 What We Are Achieving

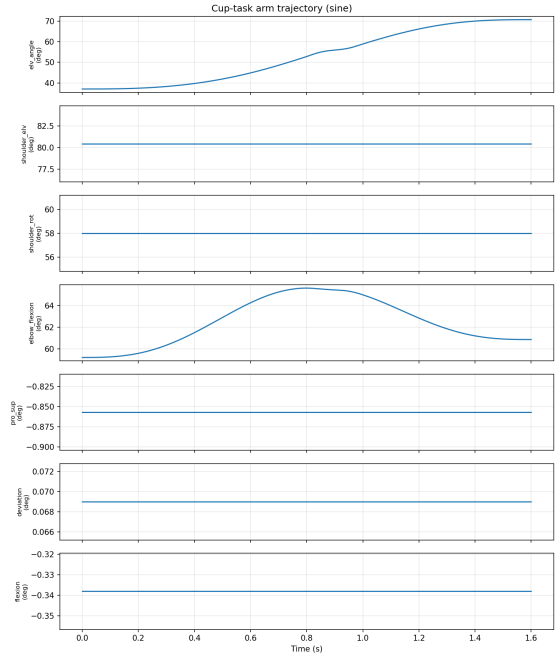
Overall goal of `compute_stiffness.py`

Generate **impedance ground truth** — a time-varying endpoint stiffness ellipsoid $\boldsymbol{K}_e(t)$ that encodes the human-like mechanical behaviour of the arm during the cup-carry task. This serves three roles in the pipeline:

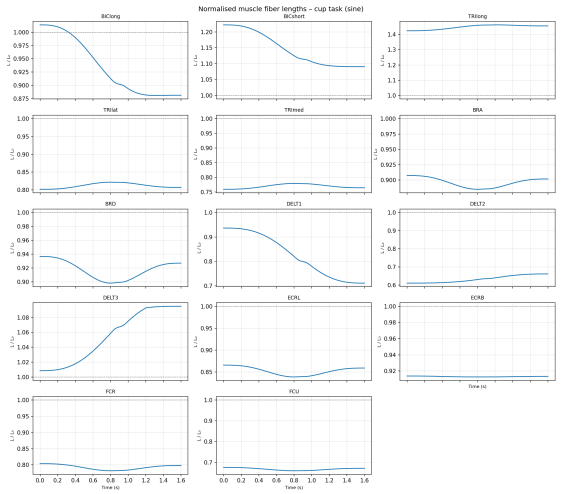
1. **Tele-Impedance reference**: $\boldsymbol{K}_e(t)$ is the target impedance that the SEA elbow exosuit must replicate via cable pre-tension c_{exo}. The null-space proxy $P_{\text{null}}(t)$ is the scalar signal that maps directly to c_{exo} in the Ajoudani et al. (2012) framework.
2. **RL reward shaping**: The PPO policy in `rl/train_ppo_residual.py` is rewarded for keeping the slave robot's impedance close to $\boldsymbol{K}_e(t)$ from the healthy reference.
3. **Perturbation robustness analysis**: The stiffness ellipse reveals in which workspace directions the arm is most vulnerable ($\lambda_{\min}$ direction) and most robust ($\lambda_{\max}$ direction), guiding the exosuit's pre-tension axis orientation.

The complete chain from arm motion to exosuit control is:

$$\underbrace{\boldsymbol{q}(t)}_{\text{IK traj.}} \xrightarrow{\text{ID}} \boldsymbol{\tau}(t) \xrightarrow{\text{SO}} \boldsymbol{a}(t) \xrightarrow{\boldsymbol{K}_e = \boldsymbol{J}^{+\top}\boldsymbol{R}\,\text{diag}(k_m)\boldsymbol{R}^{\top}\boldsymbol{J}^{+}} \underbrace{\boldsymbol{K}_e(t)}_{\text{impedance}} \xrightarrow{P_{\text{null}} = \min a_m} \underbrace{c_{\text{exo}}(t)}_{\text{SEA setpoint}} \tag{38}$$



(a) IK-solved trajectory $q(t)$ used as input to `run_stiffness()`. The five locked DOFs are flat lines; `elv_angle` and `elbow_flexion` sweep through the reaching motion.



(b) Normalised fiber lengths l_m/L_{opt} with CMC activations. Most muscles operate near $\tilde{l} \approx 1.0$–1.3 (ascending or slightly descending limb of the FL curve). TRIlong starts above 1.3 (biarticular, stretched at neutral elbow).

Figure 7: Trajectory and fiber length inputs to the stiffness computation.

7 Normalisation: Why l_{opt} and not Neutral-Pose Length

Important — normalisation reference

All fiber lengths are normalised by the muscle's *optimal fiber length* l_{opt} (a model constant from `getOptimalFiberLength()`), *not* by the neutral-pose passive length used previously.

Normalisation formula

$$\tilde{l}_m(t) = \frac{l_m(t)}{l_{\text{opt},m}} \tag{39}$$

- $\tilde{l} = 1.0$: muscle at optimal length $\Rightarrow$ maximum force
- $\tilde{l} < 1.0$: ascending limb (shortened)
- $\tilde{l} > 1.0$: descending limb / passively stretched

Why the old normalisation was wrong for TRIlong: TRIlong is biarticular (crosses both shoulder and elbow). At the neutral posture ($\varphi_{\text{sev}} \approx 80°$, $\varphi_{\text{ef}} \approx 65.6°$) it is already passively stretched well beyond l_{opt}. Using neutral-pose length as reference compressed the entire trajectory's variation onto a near-flat line near 1.0, making the normalised output meaningless. With l_{opt} as reference, the ascending/descending limb position is physiologically interpretable.

A diagnostic table is printed on each run:

Neutral-pose diagnostic (printed by run_msk and run_active_msk)

```
Muscle          l_neutral (m)   l_opt (m)   ratio
BIClong            0.1162         0.1157      1.004
TRIlong            0.1621         0.1270      1.277   <- FAR FROM OPTIMAL
...
```

Muscles with $|\tilde{l}_{\text{neutral}} - 1| > 0.35$ are flagged `FAR FROM OPTIMAL`, alerting the user that the neutral posture is on the descending limb of the force–length curve for that muscle.

8 Object-Oriented Architecture and `config.py`

The pipeline has been refactored so that each script exposes a single top-level class wrapping its end-to-end behaviour, and all simulation-wide constants live in a centralised `config.py`. The legacy module-level functions (`build_trajectory`, `run_msk`, `compute_moment_arms`, `so_frame`, ...) remain importable unchanged so that earlier sections of this document still apply verbatim.

8.1 config.py — Single Source of Truth

config.py aggregates every numerical constant used across the pipeline. Values are exposed in two interchangeable ways:

- Flat module-level constants (backwards-compatible):
  `from config import FS, MS_LABELS, ACTIVE_DOFS, NEUTRAL, MODEL_PATH, BASELINE_ALPHA, COCONTRACTION_ALPHA, FL_GAMMA, BETA_D, V_MAX_FACTOR, KP_PAPER, KD_PAPER, ...`
- Frozen SimConfig dataclass (preferred for new code):
  `from config import CFG; fs = CFG.kinematics.fs; kp = CFG.razavian2021.kp`

The dataclass groups constants into logical bundles: Paths, Kinematics, Muscles, Hill, Perturbation, Razavian2021. Running python config.py dumps the full configuration as JSON for inspection.

Centralised constants overview:

Group	Constants	Used by
Paths	MODEL_PATH, DEMO_OUTPUT_DIR	all scripts
Kinematics	FS=100 Hz, ACTIVE_DOFS (7), ACTIVE_IK_DOFS (2), NEUTRAL	all scripts
Muscles	MS_LABELS (14), HAND_BODY, HAND_LOCAL_PT	all scripts
Hill	BASELINE_ALPHA=0.02, COCONTRACTION_ALPHA=0.15, FL_GAMMA=0.45, BETA_D=0.3, V_MAX_FACTOR=10	ArmInverseDynamics, ComputeStiffness
Perturbation	T_MOVE, D_MOVE, M_EFF, T_PERT, F_PERT, SIGMA_PERT, V_DIP_FRAC, F_NEG, DT_NEG, SIG_NEG, ELV_ANGLE_BOUNDS, ELBOW_FLEX_BOUNDS	ArmCupPerturbation, ArmInverseDynamics
Razavian2021	KP_PAPER=40 N/m, KD_PAPER=50 N·s/m, FPERT_PAPER=−20 N, PERT_DUR_PAPER=20 ms	RazavianComparison

No script redeclares any of the values above; if a constant must change, it is edited in exactly one place (config.py).

8.2 Class API per Script

Each script defines exactly one public class. Calling ClassName().run() reproduces what the legacy __main__ block did. All classes can also be driven step-by-step (e.g. obj.build_trajectory() then obj.run_msk()).

Script	Class	Key methods
`arm_cup_task.py`	`ArmCupTask`	`build_trajectory, run_msk, save_outputs,`
`arm_cup_perturbation.py`	`ArmCupPerturbation`	`init_model, build_trajectory, save_trajec` `plot_all, run_msk, run`
`arm_inverse_dynamics.py`	`ArmInverseDynamics`	`run` (full ID → loop-closure → SO + K_{joint})
`compute_stiffness.py`	`ComputeStiffness`	`run` (trajectory → ID + SO → K_e, D_e tensors)
`compare_to_razavian2021.py`	`RazavianComparison`	`load_data, compute_stats, save_summary, p`

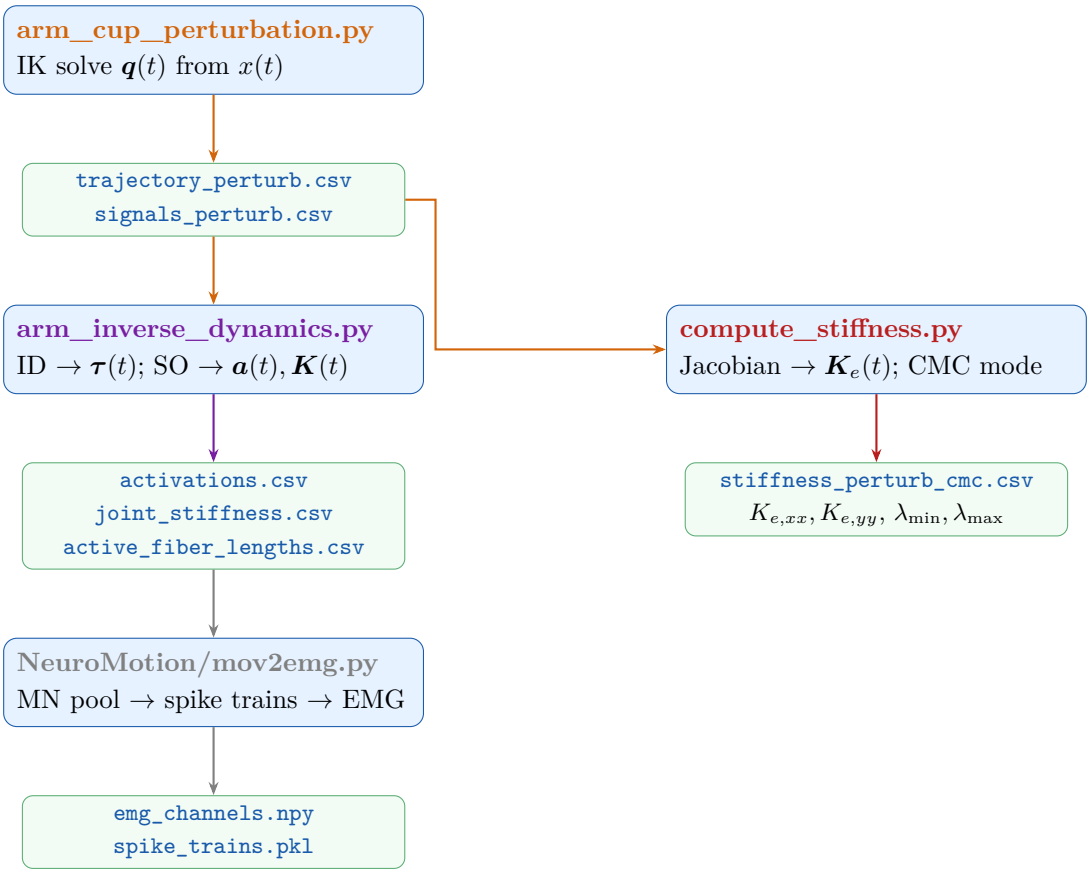
8.3 Damping Tensor $D_e(t)$

`compute_stiffness.py` now also returns endpoint damping. The Hill force–velocity slope at $v_m = 0$ gives, per muscle,

$$d_m \;=\; \beta_D \cdot a \cdot F_0 \,/\, (V_{\text{max,factor}} \cdot l_{\text{opt}}),$$

with $\beta_D = 0.3$ and $V_{\text{max,factor}} = 10\,\text{s}^{-1}$ taken from `config.Hill`. Joint-space damping is $D_{\text{joint}} = R\,\text{diag}(d_m)\,R^\top$ and the endpoint damping tensor is $D_e = J^{+\top} D_{\text{joint}} J^+$, mirroring the K_e pipeline. CSV columns `D_e_xx, D_e_yy, D_e_zz, D_e_xy, D_e_xz, D_e_yz, D_min, D_max, D_<dof>` are added alongside the existing K_e columns.

9 Complete Data Flow Summary



Signal	Symbol	Role in Tele-Impedance
Joint angles	$\boldsymbol{q}(t)$	IK → CT controller reference
Joint torques	$\boldsymbol{\tau}(t)$	ID verification / training target
Muscle activations	$\boldsymbol{a}(t)$	SO ground truth; drives NeuroMotion
Co-contraction	$\mathrm{CCI}(t)$	Null-space stiffness proxy
Joint stiffness	$\boldsymbol{K}_{\text{joint}}(t)$	Tele-Impedance Ψ mapping
Endpoint stiffness	$\boldsymbol{K}_e(t)$	Reward shaping for PPO / reference K_e
P_{null}	$\min_m a_m$	$\rightarrow c_{\text{exo}}$ SEA cable setpoint